Neural Networks: Foundations and Concepts

1. Introduction to Neural Networks

A **Neural Network** is a computational model inspired by the structure and functioning of the human brain. It consists of interconnected units called **neurons** that process information and learn patterns from data.

Neural networks are widely used in fields such as:

- Image recognition
- Natural language processing
- Speech recognition
- Autonomous driving
- Medical diagnosis

The main strength of neural networks is their ability to **learn complex patterns from large datasets**.

---

2. Biological Inspiration

Neural networks are inspired by the **human brain**.

The human brain contains billions of neurons connected through **synapses**.

Each biological neuron consists of:

Dendrites
Receive signals from other neurons.

Cell Body (Soma)
Processes the incoming signals.

Axon
Sends signals to other neurons.

Synapse
Connection between neurons where signals are transmitted.

Artificial neural networks attempt to **mimic this behavior mathematically**.

---

3. Artificial Neuron (Perceptron)

The simplest neural network unit is called a **Perceptron**.

A perceptron takes multiple inputs and produces a single output.

Components of a Perceptron

1. **Inputs (x_1, x_2, x_3, ...)**
   Features of the data.

2. **Weights (w_1, w_2, w_3, ...)**
   Importance assigned to each input.
3. **Bias (b)**
   A constant value added to shift the activation function.
4. **Activation Function**
   Determines the final output.

Mathematical Representation

The output of a neuron is:

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

Then the activation function is applied:

$$y = f(z)$$

---

4. Activation Functions

Activation functions determine whether a neuron should be activated.

Without activation functions, neural networks would behave like **simple linear models**.

4.1 Sigmoid Function

The sigmoid function maps values between **0 and 1**.

Formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Properties:

- Smooth and differentiable
- Often used for binary classification

Limitations:

- Can cause **vanishing gradient problems**

4.2 Tanh Function

The tanh function outputs values between **-1 and 1**.

Formula:

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Advantages:

- Zero-centered output

4.3 ReLU (Rectified Linear Unit)

ReLU is one of the most commonly used activation functions.

Formula:

$$f(x) = max(0, x)$$

Advantages:

- Computationally efficient
- Reduces vanishing gradient problem

---

5. Structure of Neural Networks

Neural networks are organized into **layers**.

5.1 Input Layer

The input layer receives raw data features.

Example:

If we classify handwritten digits, each pixel may represent an input feature.

5.2 Hidden Layers

Hidden layers perform intermediate computations.

Deep neural networks may contain **multiple hidden layers**.

Each hidden layer transforms data into more abstract representations.

Example:

Image recognition network:

Layer 1 → Detect edges
Layer 2 → Detect shapes
Layer 3 → Detect objects

5.3 Output Layer

The output layer produces the final prediction.

Examples:

Binary classification → 1 neuron
Multi-class classification → multiple neurons

---

6. Forward Propagation

Forward propagation is the process where **input data moves through the network** to produce predictions.

Steps:

1. Input features are fed into the network.
2. Each neuron computes a weighted sum.
3. Activation functions transform outputs.
4. The result moves to the next layer.
5. The final layer produces the prediction.

---

7. Loss Function

The **loss function** measures how wrong the model's predictions are.

The goal of training is to **minimize the loss**.

Common loss functions include:

Mean Squared Error (MSE)

Used in regression tasks.

$$MSE = \frac{1}{n}\sum(y_{true} - y_{pred})^2$$

Cross Entropy Loss

Common in classification tasks.

It measures the difference between predicted probabilities and actual labels.

---

8. Backpropagation

Backpropagation is the algorithm used to **train neural networks**.

It calculates how much each weight contributed to the error and updates it accordingly.

Steps:

1. Perform forward propagation

2. Compute loss
3. Calculate gradients of loss with respect to weights
4. Update weights using gradient descent

---

9. Gradient Descent

Gradient descent is an optimization algorithm used to **minimize the loss function**.

The idea is to adjust weights in the direction that **reduces the error**.

Weight update rule:

$$w = w - \alpha \frac{\partial L}{\partial w}$$

Where:

- **w** = weight
- **α (alpha)** = learning rate
- **L** = loss function

---

10. Learning Rate

The **learning rate** determines how big the update step is during training.

If the learning rate is:

Too small → training becomes very slow
Too large → training may become unstable

Choosing the right learning rate is critical.

---

11. Types of Neural Networks

11.1 Feedforward Neural Networks (FNN)

Data moves only **forward** from input to output.

There are **no loops** in the network.

11.2 Convolutional Neural Networks (CNN)

Used mainly for **image processing**.

CNNs use convolution layers to detect patterns like:

- Edges

- Textures
- Shapes

Applications:

- Face recognition
- Object detection

11.3 Recurrent Neural Networks (RNN)

Designed for **sequential data**.

Examples:

- Language modeling
- Speech recognition
- Time series prediction

RNNs have **memory of previous inputs**.

---

12. Overfitting and Underfitting

Overfitting

When the model memorizes training data but performs poorly on new data.

Solutions:

- Regularization
- Dropout
- More data

Underfitting

When the model is too simple to capture patterns.

Solutions:

- More complex model
- More training time

---

13. Regularization Techniques

Regularization helps prevent overfitting.

L1 Regularization

Adds penalty based on absolute weight values.

L2 Regularization

Adds penalty based on squared weight values.

---

14. Dropout

Dropout randomly **deactivates neurons during training**.

This prevents neurons from becoming too dependent on each other.

---

15. Applications of Neural Networks

Neural networks are used in many real-world systems.

Examples include:

- Self-driving cars
- Medical image analysis
- Fraud detection
- Language translation
- Recommendation systems